

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture.
(BL3)

Assessment Tool Weightage: 40%

QUESTIONS: Any 4

Total Marks:40

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

*I D Varshini / 192320007 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

ANALYTICAL PROBLEM SOLVING

1. Use Case Diagram Problem

A **Hospital Management System** allows patients to book appointments, doctors to update medical records, and administrators to manage hospital resources. The requirements are not clearly documented.

Tasks:

- a) Identify all actors involved in the system.
- b) List the primary use cases.
- c) Define relationships (include, extend, generalization).
- d) Construct a complete Use Case Diagram representing system functionality.

(10 Marks)

ANSWERS :

a) Actors involved

1. Patient – books or cancels appointments, views records and makes payments.
2. Doctor – diagnoses patients, updates records and prescribes medication.
3. Nurse – records vital signs and nursing observations.
4. Receptionist – registers patients and manages appointments.
5. Administrator – manages staff, resources and reports.
6. Pharmacist – manages prescriptions and medicine inventory.

b) Primary use cases

1. Register Patient
2. Book Appointment
3. Cancel Appointment
4. View Appointment Schedule
5. View Patient Medical Records
6. Update Medical Records
7. Manage Patient Record
8. Diagnose Patient
9. Prescribe Medication
10. Record Patient Vital Signs

11. Update Nursing Observations
12. Process Payment
13. Manage Hospital Resources
14. Manage Staff
15. Manage Medicines and Inventory
16. Manage Prescriptions
17. Generate Reports

c) Relationships

1.<<include>> Relationships

An include relationship represents a compulsory sub-function. The dashed arrow points towards the included use case.

- Book Appointment <<include>> View Appointment Schedule
- Cancel Appointment <<include>> View Appointment Schedule
- Update Medical Records <<include>> View Patient Medical Records
- Diagnose Patient <<include>> View Patient Medical Records
- Record Patient Vital Signs <<include>> View Patient Medical Records
- Manage Prescriptions <<include>> Manage Medicines and Inventory

2.<<extend>> Relationships

An extend relationship represents optional or conditional functionality. The dashed arrow points towards the base use case.

- Register Patient <<extend>> Book Appointment — when the patient is new.
- Process Payment <<extend>> Book Appointment — when payment is required.
- Prescribe Medication <<extend>> Diagnose Patient — when medication is required after diagnosis.
- Update Nursing Observations <<extend>> Record Patient Vital Signs — when additional observations are required.

3.Generalization Relationships

Generalization represents specialised forms of a broader use case. Use a solid line with a hollow triangle pointing towards the general use case.

- View Patient Medical Records —|> Manage Patient Record
- Update Medical Records —|> Manage Patient Record

d) Construction of the complete diagram

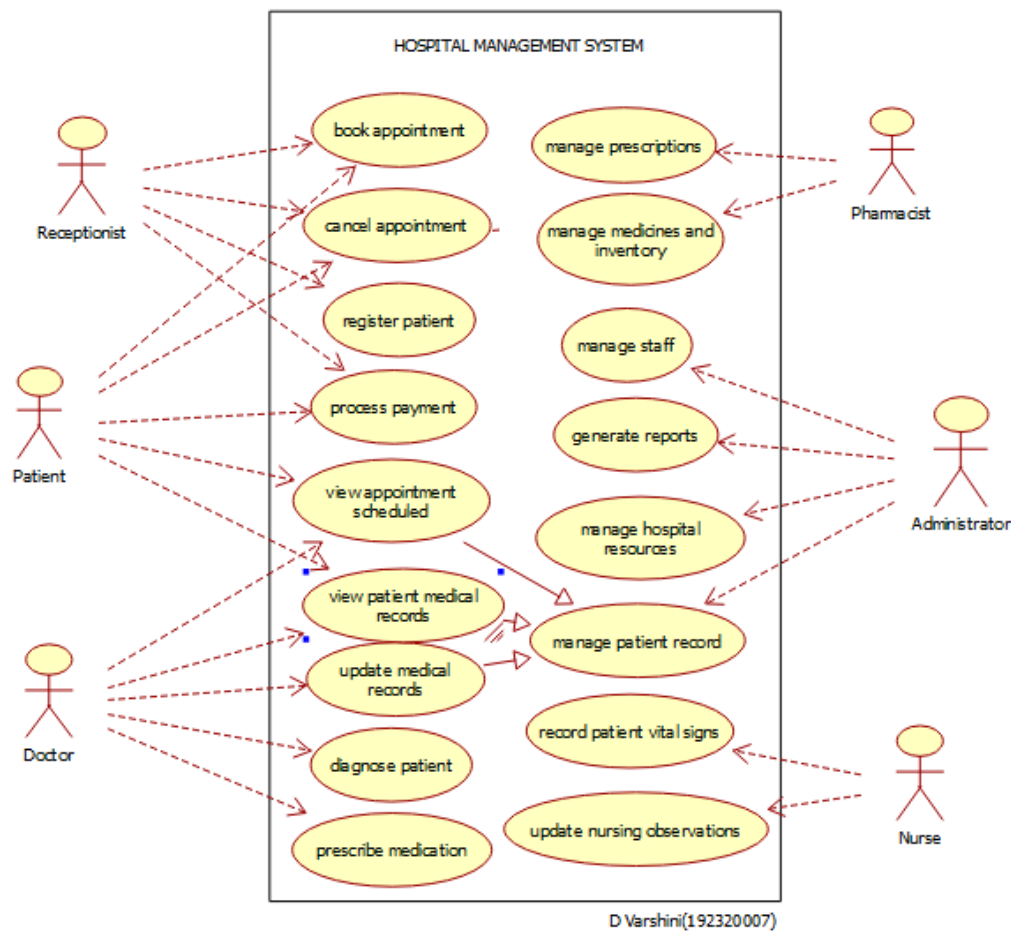


Figure 1: Use Case Diagram for the Hospital Management System, showing actors, core use cases, and include/extend/generalisation relationships.

2. Class Diagram Problem

A **Library Management System** contains entities such as Book, Member, Librarian, and IssueRecord. The relationships among entities are not properly defined.

Tasks:

- a) Identify classes with attributes and methods.
- b) Define relationships (association, aggregation, inheritance).
- c) Specify multiplicity between classes.
- d) Design a detailed Class Diagram for the system.

Answers :

The Library Management System (LMS) revolves around four core entities — Book, Member, Librarian, and IssueRecord — plus a shared superclass and a supporting Publisher entity. Analysing the described interactions (borrowing, returning, cataloguing) reveals the structural relationships between these classes.

a) Classes with Attributes and Methods

• **Person (abstract superclass):** personId, name, address, phone; getDetails(). • **Member (inherits Person):** memberId, membershipType, joinDate, maxBooksAllowed; register(), searchBook(title), requestBook(bookId), payFine(amount).

• **Librarian (inherits Person):** staffId, designation; addBook(book), removeBook(bookId), issueBook(memberId, bookId), receiveReturn(recordId).

• **Book:** bookId, title, author, isbn, category, status; checkAvailability(), updateStatus(status).

• **IssueRecord:** recordId, issueDate, dueDate, returnDate, fineAmount; calculateFine(), closeRecord().

• **Publisher:** publisherId, name; getBooksPublished().

b) Relationships

Inheritance (generalisation): Member and Librarian both inherit common identity attributes and behaviour from the abstract Person class, avoiding duplication of name/contact fields.

Association: Member and IssueRecord are associated because a member creates issue transactions; Librarian and IssueRecord are associated because a librarian

maintains/authorises each transaction; IssueRecord references a Book to know which title was borrowed.

Aggregation: Publisher and Book form a "whole-part" aggregation — a Book belongs to a Publisher's catalogue, but a Book object can conceptually exist and be catalogued independently of the Publisher object's lifecycle in the system (weak ownership), which is why an open diamond aggregation (rather than composition) is used.

c) Multiplicity

- Member (1) — IssueRecord (0..*): a member may have zero or many issue records over time.
- Librarian (1) — IssueRecord (0..*): a librarian processes many transactions. • IssueRecord (0..*) — Book (1): each record refers to exactly one book, but a book has many historical records.
- Publisher (1) — Book (0..*): a publisher publishes many books; each book has exactly one publisher.

d) Class Diagram

Class Diagram: Library Management System

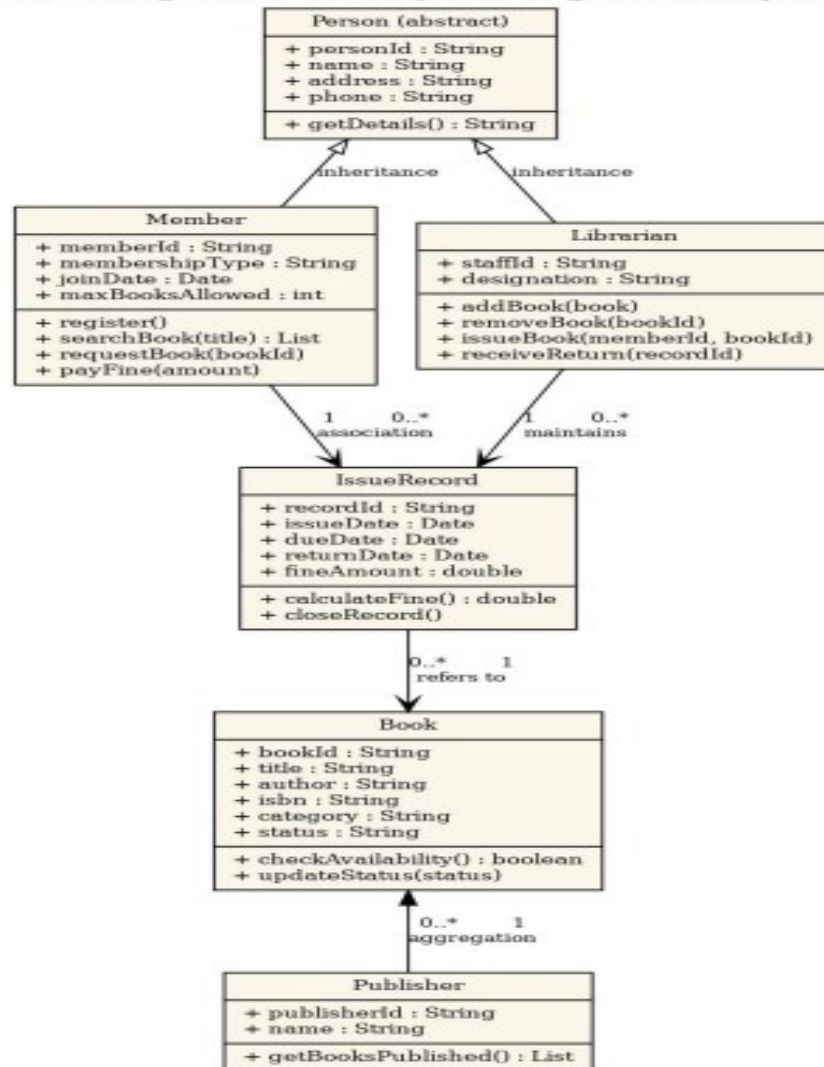


Figure 2: Class Diagram for the Library Management System, showing inheritance, association, aggregation, attributes, methods, and multiplicities.

3. Sequence Diagram Problem

In an **Online Food Ordering System**, a customer selects food items, places an order, makes payment, and receives order confirmation. The interaction flow is unclear.

Tasks:

- Identify objects involved in the process.
- Define message flow between objects.
- Represent activation and lifelines.
- Construct a Sequence Diagram showing complete interaction flow.

Answers :

The ordering process is a time-ordered interaction between a Customer and several system/ external objects. Modelling it as a sequence diagram clarifies the exact order of

messages, which object is active (has control) at each point, and where synchronous versus asynchronous calls occur.

a) Objects Involved

- **Customer** — the actor initiating the interaction.
- **:UI (App/Web)** — boundary object handling presentation and user input.
- **:OrderController** — control object that validates and coordinates the order.
- **:PaymentGateway** — external entity object that authorises payment.
- **:Restaurant System** — entity object that receives and prepares the order.

b) Message Flow

The flow begins with the customer browsing the menu and selecting items, which the UI relays to the customer as a rendered list. Placing the order triggers a synchronous call from the UI to the OrderController, which validates the order (a self-call) before returning an orderId. The UI then requests payment details from the customer, forwards them to the PaymentGateway for processing, and on confirmation, the OrderController transmits the order to the Restaurant System. Once the kitchen accepts the order, a confirmation cascades back through the controller and UI to the customer.

c) Activation and Lifelines

Each participant is represented by a vertical dashed lifeline running down the page in the order the interaction unfolds. Shaded activation bars are placed on the lifeline of whichever object is actively executing — for example, the UI object remains active for almost the full interaction since it mediates every step, while the PaymentGateway is only active during the payment

processing window. Solid arrows denote synchronous request messages and dashed arrows denote return (reply) messages.

d) Sequence Diagram

Sequence Diagram: Online Food Ordering System

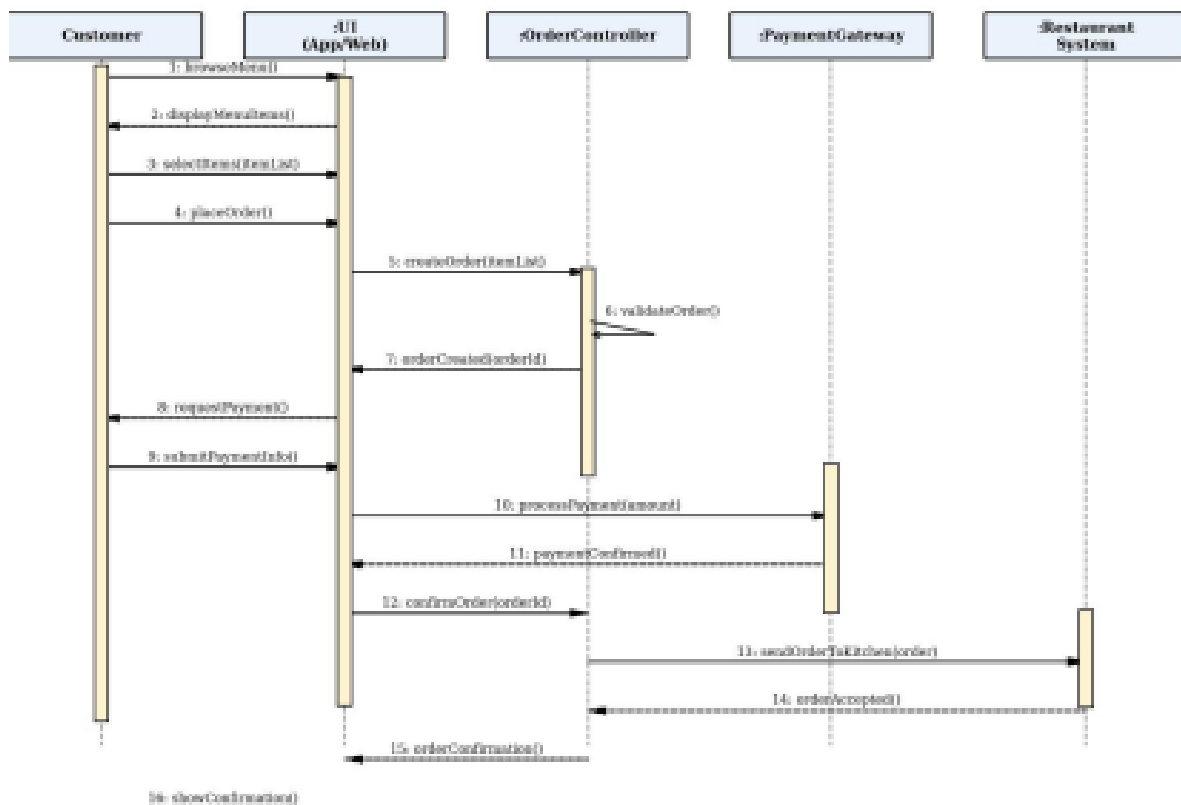


Figure 3: Sequence Diagram for the Online Food Ordering System, showing lifelines, activation bars, and the complete request/response message flow.

4. Activity Diagram Problem

A **Student Admission System** includes application submission, document verification, fee payment, and enrollment confirmation. The workflow lacks proper decision-making steps.

Tasks:

- Identify all activities in the workflow.
- Add decision nodes for approval/rejection.
- Include parallel processing where applicable.
- Design an optimized Activity Diagram.

Answer :

The admission workflow spans several stages with clear decision points (document validity, payment success) and an opportunity for concurrent processing (document verification and eligibility checking can occur in parallel once documents are uploaded).

a) Activities Identified

- Submit Application Form
- Upload Documents
- Verify Documents (parallel branch 1)
- Check Eligibility Criteria (parallel branch 2)
- Reject Application & Notify Applicant
- Pay Admission Fee
- Retry / Cancel Payment
- Generate Fee Receipt
- Confirm Enrollment
- Send Confirmation (Email/SMS)

b) Decision Nodes

Two decision diamonds drive the workflow's branching logic. The first, "*Documents Valid?*", follows the join of the parallel verification activities: a "No" outcome routes the flow to rejection and applicant notification, while "Yes" proceeds to fee payment. The second, "*Payment Successful?*", allows the applicant to retry a failed transaction (looping back to the payment activity) or, on success, proceed to receipt generation and enrolment.

c) Parallel Processing

After documents are uploaded, a *fork* node splits the flow into two concurrent activities — document verification and eligibility checking — which do not depend on each other and can therefore run simultaneously (e.g., by an admissions clerk and an academic committee at the same time). A subsequent *join* node synchronises both branches before the combined decision on document validity is made, ensuring the workflow does not proceed until both checks are complete.

d) Activity Diagram

Activity Diagram: Student Admission System

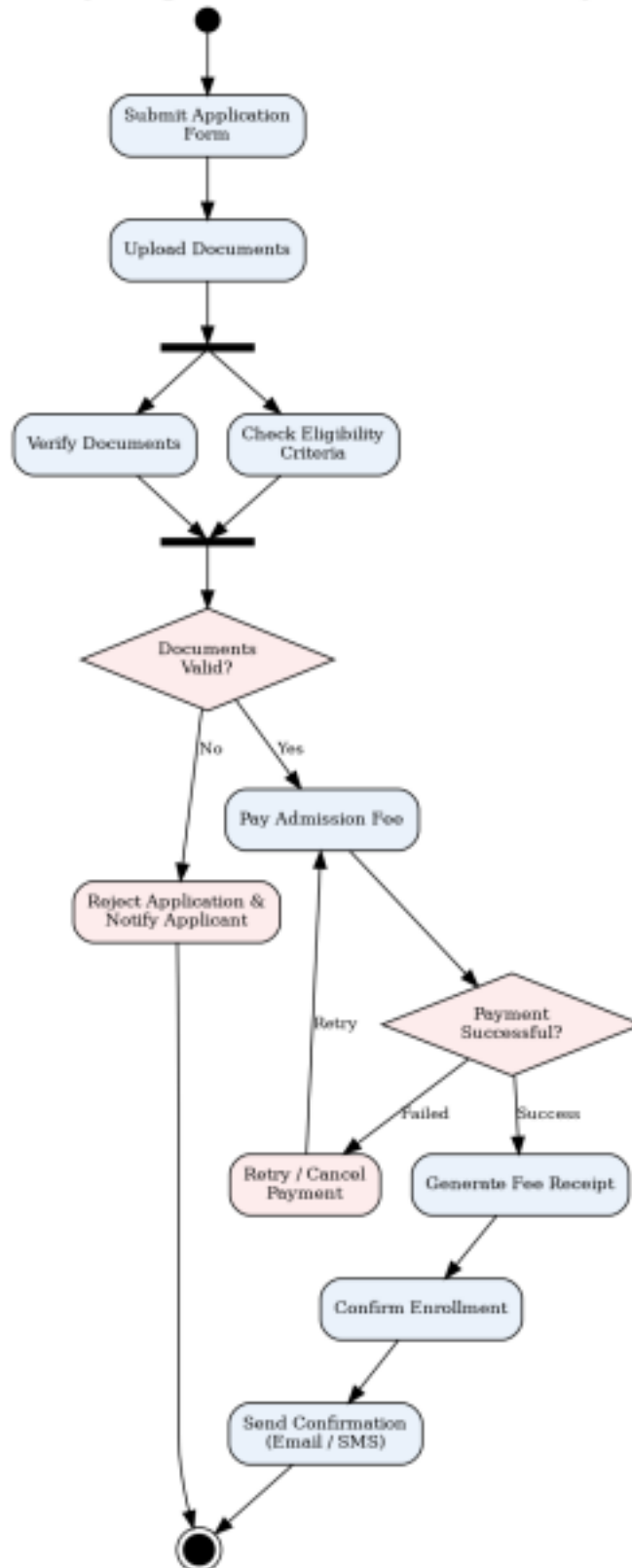


Figure 4: Activity Diagram for the Student Admission System, showing decision nodes, fork/join bars for parallel verification, and the complete workflow to enrolment.

RUBRICS FOR CALCULATION

ANALYTICAL PROBLEM SOLVING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation